

Distance Transformation in Parallel

Shyong Jian Shyu, Ting Wei Chou and Tsorng Lin Chia*
Department of Computer Science and Information Engineering,
Ming Chuan University, Taoyuan, Taiwan

*Department of Computer and Communication Engineering,
Ming Chuan University, Taoyuan, Taiwan
sjshyu@csie.mcu.edu.tw

Abstract

In this paper, we design and implement an efficient parallel algorithm for computing the distance transformation (DT) of a binary image on a PC cluster. The parallel algorithm decomposes the input image into parallelogram-like areas within which a 2-pass scanning is performed and among which a wave-front parameterized strategy is adopted to coordinate the computations and communications on the cluster. In regard to the distance transform for a 3840×3840 binary image, the speed-up ratio of our algorithm over that of the sequential approach is 8.18 by using a cluster of 16 PCs. When the time for reading raw data is included, the speed-up ratio becomes 13.44.

1 Introduction

Consider an $M \times N$ binary image I containing a set of solid objects O . We refer to the area of the image that does not belong to O as background denoted as B . That is, $O \cup B = I$ and $O \cap B = \emptyset$. For each pixel in the background, the *distance transformation* (DT, for short) is to find the shortest distance from this pixel to all of the objects. Formally, for each pixel $x \in I$, the DT of x , denoted as $dt(x)$, is defined by

$$dt(x) = \min\{\delta(x, y) \mid y \in B\}$$

where $\delta(x, y)$ is the function measuring the distance between x and y .

DT is a fundamental procedure for a wide range of applications in image matching and shape analysis, such as the measurement of trabecular bone thickness

[13], vehicle path planning [7], [8], medial axis [11], Chinese words recognition [14], [15], cracks rendering [18] or ray-tracing [17], and so on.

Depending upon various applications, the *distance function* may be defined in different ways. Some most important distance functions are octagonal [5], weighted (chamfer) [1], and Euclidean [2], to name a few. In this paper, we are interested in finding the chamfer distances map of a binary image I .

Consider two pixels x and y in I . Let r and s be the isothetic and diagonal moves from x to y respectively. Obviously, there are many ways to move from x to y . Let ρ be a certain sequence of moves from x to y . Let a and b be two independent weights for the isothetic and diagonal moves respectively. We define $l_{a,b}$ as

$$l_{a,b}(\rho) = ra + sb.$$

Then the (a, b) chamfer distance between x and y is defined as follows:

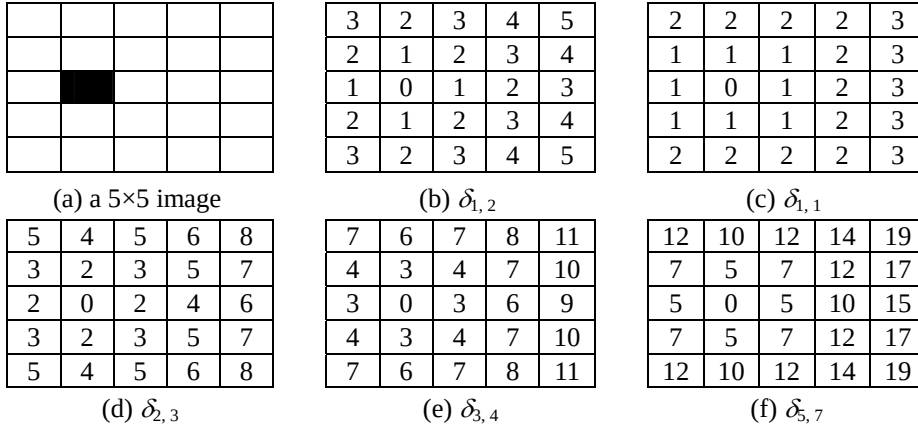
$$\delta_{a,b}(x, y) = \min_{\rho} \{l_{a,b}(\rho)\}.$$

When $(a, b) = (1, 2)$ ((1, 1), (2, 3), (3, 4) or (5, 7)), $\delta_{1,2}(x, y)$ ($\delta_{1,1}(x, y)$, $\delta_{2,3}(x, y)$, $\delta_{3,4}(x, y)$ or $\delta_{5,7}(x, y)$ respectively) is called city block distance or Manhattan (chess board, chamfer 2-3, chamfer 3-4 or chamfer 5-7 respectively) distance between x and y . Table 1 summarizes some popular usages of (a, b) 's.

Figure 1 shows the results of different settings of

Table 1. Popular chamfer parameters (a, b) 's

	city block	chess board	chamfer 2-3	chamfer 3-4	chamfer 5-7
a	1	1	2	3	5
b	2	1	3	4	7


 Figure 1. Results of chamfer distances by different settings of (a, b) 's.

(a, b) 's for a certain 5×5 image where the black pixel in Figure 1 (a) is an object.

2 Previous Works

Many algorithms were developed for computing DT [2], [3], [4], [6], [10], [12], [17]. Rosenfeld and Pfaltz [12] designed a 2-pass DT algorithm, while Chen and Yang [3] proposed a 4-pass DT algorithm and both of the algorithms cost $O(MN)$ computation time for an $M \times N$ binary image. These two algorithms scan the whole image pixel by pixel for 2 and 4 times, respectively.

The 2-pass DT algorithm [12] includes two raster scan, namely the *forward pass* and the *backward pass*, on the given binary image I . Initially, all of the pixels belonging to the objects are assigned as 0 and all pixels of the background are assigned as ∞ . The forward pass is a row-by-row computation on I from left to right and from top to bottom to find the partial distance value $d_1(I(i, j))$ for each pixel $I(i, j)$ where $d_1(I(i, j))$ is calculated in terms of

$$d_1(I(i, j)) = \min \left\{ \begin{array}{l} d_1(I(i, j)), \\ d_1(I(i-1, j-1)) + a, \\ d_1(I(i-1, j)) + b, \\ d_1(I(i-1, j+1)) + a, \\ d_1(I(i, j-1)) + b, \end{array} \right\}.$$

Likewise, the backward pass is executed from right to left and from bottom to top to find the second

partial distance value $d_2(I(i, j))$ for each pixel $I(i, j)$ calculated in terms of

$$d_2(I(i, j)) = \min \left\{ \begin{array}{l} d_2(I(i, j)), \\ d_2(I(i+1, j+1)) + a, \\ d_2(I(i+1, j)) + b, \\ d_2(I(i+1, j-1)) + a, \\ d_2(I(i, j+1)) + b, \end{array} \right\}.$$

Figure 2 (a) and (b) reveals the four neighbors of $I(i, j)$ in computing d_1 and d_2 corresponding to the forward and backward passes respectively.

$I(i-1, j-1)$	$I(i-1, j)$	$I(i-1, j+1)$
$I(i, j-1)$	$I(i, j)$	

 (a) the neighbors of $I(i, j)$ in forward pass

	$I(i, j)$	$I(i, j+1)$
$I(i+1, j-1)$	$I(i+1, j)$	$I(i+1, j+1)$

 (b) the neighbors of $I(i, j)$ in backward pass

 Figure 2. The neighbors of $I(i, j)$ in the 2-pass DT algorithm.

It is not hard to realize that the chamfer distance of $I(i, j)$ is

$$dt(I(i, j)) = \min(d_1(i, j), d_2(i, j)).$$

As compared to the 2-pass algorithm, the 4-pass DT algorithm [3] reduces the computations of each pixel by choosing less neighbors, but increases the passes into 4 for scanning the image which involves: (1) from top-left to bottom-right, (2) from top-right to

bottom-left, (3) from bottom-right to top-left, (4) from bottom-left to top-right. The neighbors of $I(i, j)$ corresponding to the 4-pass scans are shown in Figure 3.

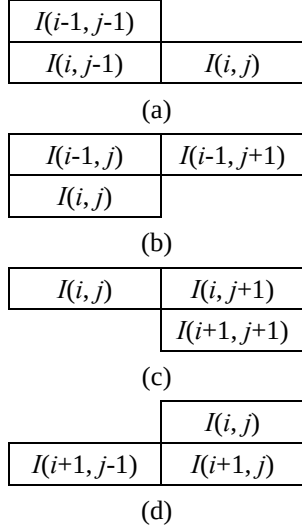


Figure 3. The neighbors of $I(i, j)$ in the 4-pass DT algorithm.

The above algorithms were designed and executed in sequential. Bruno and Costa [2] presented a dilations-based DT algorithm. For each pixel $I(i, j)$ in B , their algorithm computes the least distance from $I(i, j)$ to each of the pixels belonging to O . Thus, the larger the areas of objects occupy in I , the larger the execution time costs. The worst case of time complexity is $O(M^2 N^2)$. Since it computes the distances directly among each pair of the pixels (i, j) where $x \in B$ and $y \in O$, the most critical advantage of this algorithm is that it can be tailored to compute all kinds of distances exactly, including Euclidean, chamfer, and so on. The algorithm implemented in parallel by distributing all object pixels evenly onto four personal computers (PCs) which are connected by Ethernet. The performance varies depending on the number of object pixels in the given image.

3 The Proposed Parallel DT Algorithm

The data needed in the computations of $d_1(I(i, j))$

and $d_2(I(i, j))$ are the four specified neighbors of $I(i, j)$ (see Figure 2) in the 2-pass algorithm. This idea is inherited parallel. The divide-and-conquer strategy in designing parallel algorithms can be applied easily. An image can be decomposed into a certain number of partitions proportional to the number of processing units and each partition is distributed onto a certain PC. Then the distance transformation of all partitions can be computed simultaneously in all PCs in the cluster. Yet the data dependency in the 2-pass algorithm that the distance is correct only when both passes are finished prevents the trivial divide-and-conquer approach from achieving highly parallel performance. The parallel DT algorithm is designed based upon the 2-pass algorithm with the following three principles:

- Each partition is parallelogram-like.
- The size of a partition should be a tradeoff among the performances of computation ability of the PCs and the communication efficiency of the connection network which are affected by the number of PCs in the cluster and the size of the input image.
- We shall make the processor idle time as less as possible.

Figure 4 illustrates the reason why we adopt the first principle. As shown in Figure 4. The neighbors of a certain set of pixels within a parallelogram constitute another parallelogram. The other two principles are easily understood for obtaining a better performance in parallel computing.

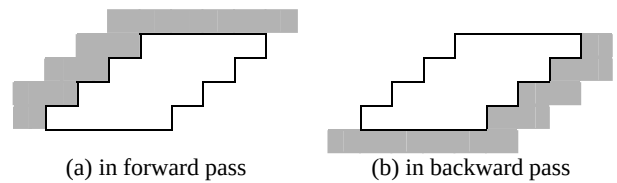


Figure 4. Parallelogram partitions and their neighbors.

We set the size of a parallelogram partition to be $\alpha \times \beta$. Then we obtain $(M/\alpha) \times (N/\beta)$ partitions for an $M \times N$

input image. The data partition of the whole image is shown in Figure 5 where we assume a p -PCs, named as $P_1, P_2, \dots, P_p$, cluster is our test platform. When a PC, say P_i , at partition $[i, j]$ where i and j are the index of the partitions finishes the computations within the partition, it continues to execute the computations of partition $[i, j+1]$ after it sends the results at the bottom row of partition $[i, j]$ (the gray area in Figure 5) down to partition $[i+1, j-1]$ where P_{i+1} is ready to proceed. In the first iteration, only P_1 is executing which processes a half partition only. In fact, in the first iteration of each PC, it faces a half partition.

Figure 6 illustrates a simple example. It indicates an image which is decomposed into 36 partitions. Figures 7 (a) and (b) show the time-space diagrams of the forward passes by using 2-PCs and 4-PCs clusters respectively in the proposed algorithm. It can easily to see that the execution time is shortened by using 4-PCs cluster as compared to 2-PCs cluster. Note that we do not consider the communication issue so far in Figure 7.

In regard to a specific PC in the cluster, the time complexity of the proposed algorithm is $O(MN/p)$ for computations and $O(MN/\alpha\beta p)$ for communications. Note that the time complexity of computations in Rosenfeld and Pfaltz's [12] and Chen and Yang's [3] approaches are also $O(MN/p)$. Their communications models are different from ours and they did not conduct computational experiments explicitly.

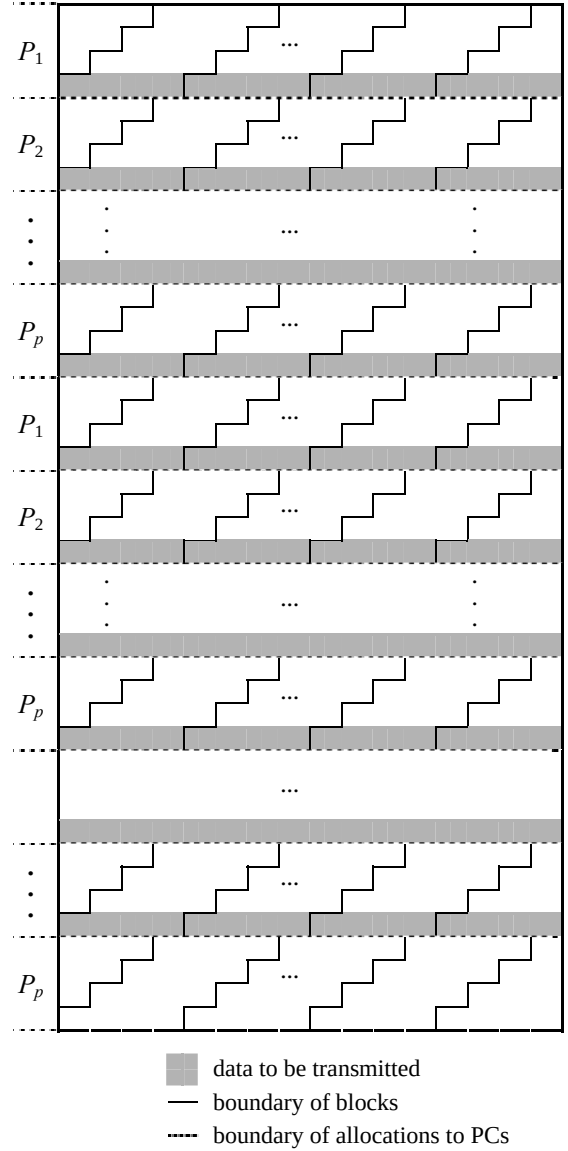


Figure 5. The proposed data partition strategy.

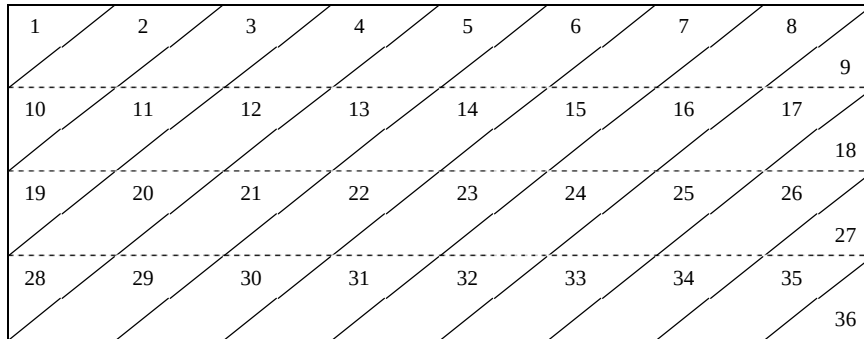
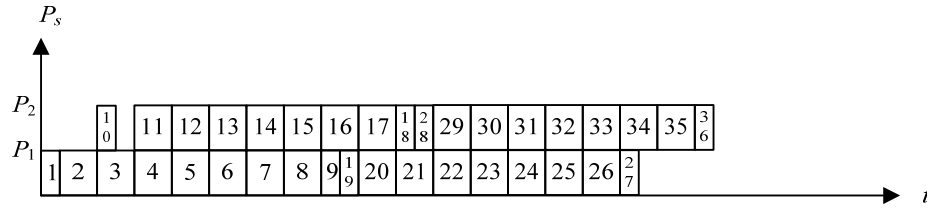
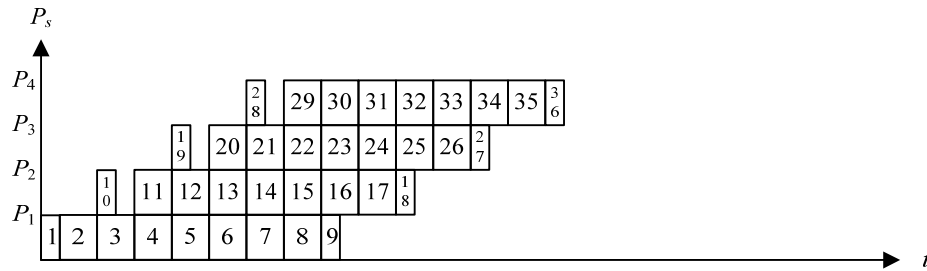


Figure 6. An image with 36 decomposed parallelogram partitions.



(a) the time-space diagram of Figure 6 in the forward pass on 2 PCs



(b) the time-space diagram of Figure 6 in the forward pass on 4 PCs

Figure 7. An example to illustrate the time-space diagrams of Figure 6 in the forward passes of using 2- and 4-PC clusters.

4 Experimental Results

To realize the performance of our parallel algorithm on a PC cluster, our test platform is an 16-PCs cluster connected via Ethernet. Each PC runs Red-Hat Linux with an AMD Athlon 2200+ CPU and 512 MB RAM. The parallel environment is configured by using *parallel virtual machine* (PVM). We also use a *network files system* (NFS) as a sharing memory for raw data. The NFS can efficiently reduce the redundancy of reading the raw data.

Given a 3840×3840 binary image, we computed the (1, 1) chamfer distance transformation of the image by using the parallel algorithm with $\alpha = \beta = 3840/(2p)$ where $p \geq 2$.

Also, the sequential 2-pass algorithm [1] was

implemented in one of the PC in our cluster for the comparison purpose. Table 2 summarizes the execution time needed in the computations and the necessary communications spending in the calculation of DT. Figure 8 depicts the results of the speedup ratios of our algorithm, measured as the ratio of the over that by our parallel algorithm. The curve shows the scalability of our parallel algorithm. Note that the time needed in reading the raw data into the PC(s) is omitted in Table 2 for both of the sequential and the purposed parallel algorithms. Consider a certain PC in the cluster when adopting the parallel algorithm. Since only the data belonging to the partitions needs to be read into this PC, that is, there

Table 2. Results of total time needed by our parallel algorithm and the 2-pass sequential algorithm [1]

Time (Sec.)	sequential 2-pass algorithm (<i>seq</i>)		1.52					
	Our parallel algorithm (<i>par</i>)	$p = 2$	$p = 4$	$p = 6$	$p = 8$	$p = 10$	$p = 12$	$p = 16$
		0.96	0.55	0.39	0.30	0.24	0.22	0.19
Speedup Ratio (<i>seq/par</i>)		1.58	2.76	3.94	5.08	6.22	6.82	8.18

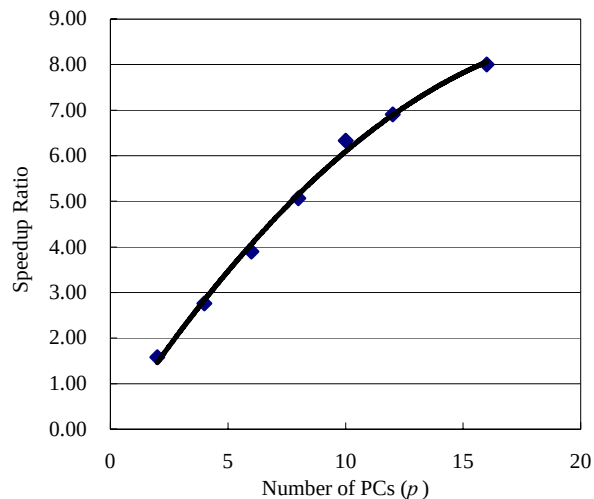


Figure 8. Speed-up ratios of purposed parallel algorithm

Table 3. Results of total time needed by our parallel algorithm and the 2-pass sequential algorithm [1] including reading data

Time (Sec.)	sequential 2-pass algorithm (<i>seq</i>)	6.95						
	Our parallel algorithm (<i>par</i>)	$p = 2$	$p = 4$	$p = 6$	$p = 8$	$p = 10$	$p = 12$	$p = 16$
		3.52	1.86	1.25	0.95	0.76	0.66	0.52
	Speedup Ratio (<i>seq/par</i>)	1.97	3.74	5.54	7.32	9.09	10.46	13.44

is no need to read the whole image for this PC, the time needed by data reading can be reduced in parallel. The time results with various p including the data reading are summarized in Table 3.

5 Conclusions

In this paper, we design an efficient parallel 2-pass DT algorithm which is suitable to run in a PC cluster. Based upon the computational experimental result, our parallel design is efficient and scalability. In the near future, we shall examine the proposed parallel algorithm by extensive experiments, including different settings of α and β , just to name a few, in order to verify the robustness of our parallel algorithm. Also, it is interesting to test 4-pass algorithm or any other traditional DT algorithms.

References

- [1] G. Borgefors, "Distance transformations in digital images," *CVGIP*, Vol. 34, No. 3, pp. 344-371, 1986.
- [2] O. M. Bruno and L. D. F. Costa, "A parallel implementation of exact Euclidean distance transform based on exact dilations," *Microprocessors and Microsystems*, Vol. 28, Issue 3, pp. 107-113, 2004.
- [3] C. H. Chen and D. L. Yang, "Fast algorithm and its systolic realization for distance transformation," *IEE Proc. on Computers and Digital Techniques*, Vol. 143, No. 3, pp. 168-173, 1996.
- [4] T. L. Chia, K. B. Wang, Z. Chen, and D. C. Lou, "Parallel distance transforms on a linear array architecture," *Information Processing Letters*, Vol. 82, No. 2, pp. 73-81, 2002.
- [5] P. P. Das, "Best simple octagonal distances in digital geometry," *Approximation Theory*, Vol. 68, Issue 2, pp.

- 155-174, 1992.
- [6] A. Forsmo, "The distance transform algorithm on a two-processor computer," *ICIAP '99*, pp. 27-29, 1999.
 - [7] J. H. Hornga and J. T. Li, "Vehicle path planning by using adaptive constrained distance transformation," *Pattern Recognition*, Vol. 35, pp. 1327-1337, 2002.
 - [8] S. O. Joon, H. C. Yoon, B. P. Jin, and Y. F. Zheng, "Complete coverage navigation of cleaning robots using triangular – cell - based map," *IEEE Trans. on Industrial Electronics*, Vol. 51, No. 3, pp. 718-726, 2004.
 - [9] U. Montanari, "A method for obtaining skeletons using a quasi-euclidean distance," *JACM*, Vol. 15, Issue 4, pp. 600-624, 1968.
 - [10] Y. Pan, Y. Li, J. Li, K. Li, and S. Q. Zheng, "Efficient parallel algorithms for distance maps of 2D binary images using an optical bus," *IEEE Trans. on Systems, Man and Cybernetics, Part A*, Vol. 32, No. 2, pp. 228-236, 2002.
 - [11] E. Remy and E. Thiel, "Medial axis for chamfer distances: computing look-up tables and neighbourhoods in 2D or 3D," *Pattern Recognition Letters*, Vol. 23, Issue 6, pp. 649-661, 2002.
 - [12] A. Rosenfeld and J. L. Pfaltz, "Distance functions on digital pictures," *Pattern Recognition*, Vol. 1, pp. 33-61, 1968.
 - [13] P. K. Saha and F. W. Wehrli, "Measurement of trabecular bone thickness in the limited resolution regime of in vivo MRI by fuzzy distance transform," *IEEE Trans. on Medical Imaging*, Vol. 23, No. 1, pp. 53-62, 2004.
 - [14] D. Shi, R. I. Damper and S. R. Gunn, "Handwritten Chinese radical recognition using nonlinear active shape models," *IEEE Trans. on Pattern Analysis and Machine Intelligence*, Vol. 25, Issue 2, pp. 277-280, 2003.
 - [15] D. Shi, R. I. Damper and S. R. Gunn, "Offline handwritten Chinese character recognition by radical decomposition," *TALIP, ACM*, Vol. 2 Issue 1, pp. 27-48, 2003.
 - [16] F. Y. Shih, T. K. Chung and C. C. Pu, "Pipeline architectures for recursive morphological operations," *IEEE Trans. on Image Processing*, Vol. 4, No.1, pp. 11-18, 1995.
 - [17] M. Sramek and A. Kaufman, "Fast ray-tracing of rectilinear volume data using distance transforms," *IEEE Trans. on Visualization and Computer Graphics*, Vol. 6, No. 3, pp. 236-252, 2000.
 - [18] B. Wyvill, K. V. Overveld and S. Carpendale, "Rendering cracks in Batik," *NPAR '04, ACM*, pp. 61-149, 2004.